Name: Savitha R

Reg.no: 192421368

10.Write a Python Program to Implement Expectation & Maximization Algorithm

Program:

```
# Expectation Maximization (EM) Algorithm
# Simple implementation using Python

import math

# Dataset
data = [2, 3, 4, 5, 10, 11, 12, 13]

# Number of clusters
k = 2

# Initial means
means = [3.0, 11.0]

# Initial variances
variances = [1.0, 1.0]

# Initial probabilities
weights = [0.5, 0.5]

# Number of iterations
iterations = 10
```

```
# Gaussian Probability Density Function
def gaussian(x, mean, variance):
    return (1 / math.sqrt(2 * math.pi * variance)) * \
           math.exp(-((x - mean) ** 2) / (2 * variance))


# EM Algorithm
for iteration in range(iterations):

    # -------------------------
    # E-STEP
    # -------------------------

    responsibilities = []

    for x in data:

        probabilities = []

        for j in range(k):
            probability = weights[j] * gaussian(
                x,
                means[j],
                variances[j]
            )
            probabilities.append(probability)

        total = sum(probabilities)
```

```
    responsibility = [
        p / total for p in probabilities
    ]

    responsibilities.append(responsibility)


# -------------------------
# M-STEP
# -------------------------

for j in range(k):

    # Calculate number of points belonging
    # to cluster j
    N = sum(
        responsibilities[i][j]
        for i in range(len(data))
    )

    # Update mean
    means[j] = sum(
        responsibilities[i][j] * data[i]
        for i in range(len(data))
    ) / N

    # Update variance
    variances[j] = sum(
        responsibilities[i][j] *
        (data[i] - means[j]) ** 2
```

```
            for i in range(len(data))
        ) / N

        # Update weight
        weights[j] = N / len(data)


    # Display results
    print("\nIteration:", iteration + 1)

    print("Means      :", [round(x, 4) for x in means])
    print("Variances  :", [round(x, 4) for x in variances])
    print("Weights    :", [round(x, 4) for x in weights])


# -------------------------
# FINAL CLUSTER ASSIGNMENT
# -------------------------

print("\nFinal Cluster Assignment")
print("-------------------------")

for i in range(len(data)):

    cluster = responsibilities[i].index(
        max(responsibilities[i])
    )

    print(
        "Data:",
```

```
        data[i],
        "-> Cluster:",
        cluster + 1
    )
```

Output:

```
>>
================ RESTART: E:/Machine Learning/experiment 10lab.py =============

Iteration: 1
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 2
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 3
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 4
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]
```

```
Iteration: 5
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 6
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 7
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 8
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 9
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]
```

```
Iteration: 9
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Iteration: 10
Means      : [3.5, 11.5]
Variances  : [1.25, 1.25]
Weights    : [0.5, 0.5]

Final Cluster Assignment
------------------------
Data: 2 -> Cluster: 1
Data: 3 -> Cluster: 1
Data: 4 -> Cluster: 1
Data: 5 -> Cluster: 1
Data: 10 -> Cluster: 2
Data: 11 -> Cluster: 2
Data: 12 -> Cluster: 2
Data: 13 -> Cluster: 2
```